

natural language texts and knowledge bases (KB). The size of the training dataset is also presented in the figure. We next elaborate on the benchmark tasks and their corresponding baselines.

MNIST-R: A Synthetic MNIST Test Suite. This benchmark is designed to test various features of Scallop such as negation and aggregation. Each task takes as input one or more images of hand-written digits from the MNIST dataset [Lecun et al. 1998] and performs simple arithmetic (sum2, sum3, sum4), comparison (less-than), negation (not-3-or-4), or counting (count-3, count-3-or-4) over the depicted digits. For count-3 and count-3-or-4, we count digits from a set of 8 images. For this test suite, we use a CNN-based model, DeepProbLog (DPL) [Manhaeve et al. 2021], and our prior work [Huang et al. 2021] (Prior) as the baselines.

HWF: Hand-Written Formula Parsing and Evaluation. HWF, proposed in [Li et al. 2020], concerns parsing and evaluating hand-written formulas. The formula is provided in the form of a sequence of images, where each image represents either a digit (0-9) or an arithmetic symbol (+, −, ×, ÷). Formulas are well-formed according to a grammar and do not divide by zero. The size of the formulas ranges from 1 to 7 and is indicated as part of the input. The goal is to evaluate the formula to obtain a rational number as the result. We choose from [Li et al. 2020] the baselines NGS-*m*-BS, NGS-RL, and NGS-MAPO, which are *neurosymbolic methods* designed specifically for this task.

Pathfinder: Image Classification with Long-Range Dependency. In this task from [Tay et al. 2020], the input is an image containing two dots that are possibly connected by curved and dashed lines. The goal is to tell whether the dots are connected. There are two subtasks, Path and Path-X, where Path contains 32×32 images and Path-X contains 128×128 ones. We pick as baselines standard CNN and Transformer based models, as well as the state-of-the-art neural models S4 [Gu et al. 2021], S4* [Gu et al. 2022], and SGConv [Li et al. 2022].

PacMan-Maze: Playing PacMan Maze Game. As presented in Section 2, this task tests an agent’s ability to recognize entities in an image and plan the path for the PacMan to reach the goal. An RL environment provides the game state image as input and the agent must plan the optimal action {up, down, left, right} to take at each step. There is no “training dataset” as the environment is randomized for every session. We pick as baseline a CNN based Deep-Q-Network (DQN). Unlike other tasks, we use the “success rate” metric for evaluation, i.e., among 1000 game sessions, we measure the number of times the PacMan reaches the goal within a certain time-budget.

CLUTRR: Kinship Reasoning from Natural Language Context. In this task from [Sinha et al. 2019], the input contains a natural language (NL) passage about a set of characters. Each sentence in the passage hints at kinship relations. The goal is to infer the relationship between a given pair of characters. The target relation is not stated explicitly in the passage and it must be deduced through a reasoning chain. Our baseline models include RoBERTa [Liu et al. 2019], BiLSTM [Graves et al. 2013], GPT-3-FT (fine-tuned), GPT-3-ZS (zero-shot), and GPT-3-FS (5-shot) [Brown et al. 2020]. In an alternative setup, CLUTRR-G, instead of the NL passage, the structured kinship graph corresponding to the NL passage is provided, making it a *Knowledge Graph Reasoning* problem. For CLUTRR-G, we pick GAT [Veličković et al. 2017] and CTP [Minervini et al. 2020] as baselines.

Mugen: Video-Text Alignment and Retrieval. Mugen [Hayes et al. 2022] is based on a game called CoinRun [Cobbe et al. 2019]. In the video-text alignment task, the input contains a 3.2 second long video of gameplay footage and a short NL paragraph describing events happening in the video. The goal is to compute a similarity score representing how “aligned” they are. There are two subsequent tasks, Video-to-Text Retrieval (VTR) and Text-to-Video Retrieval (TVR). In TVR, the input is a piece of text and a set of 16 videos, and the goal is to retrieve the video that best aligns with the text. In VTR, the goal is to retrieve text from video. We compare our method with SDSC [Hayes et al. 2022].

CLEVR: Compositional Language and Elementary Visual Reasoning [Johnson et al. 2017]. In this visual question answering (VQA) task, the input contains a rendered image of geometric objects and a NL question that asks about counts, attributes, and relationships of objects. The goal is to

Task	Input	Neural Net	Interface Relation(s)	Scallop Program	Features			LoC
					R	N	A	
MNIST-R	Images	CNN	<code>digit(id, digit)</code>	Arithmetic, comparison, negation, and counting.		✓	✓	2 [†]
HWF	Images	CNN	<code>symbol(id, symbol)</code> <code>length(len)</code>	Parses and evaluates formula over recognized symbols.	✓			39
Pathfinder	Image	CNN	<code>dot(id)</code> <code>dash(from_id, to_id)</code>	Checks if the dots are connected by dashes.	✓			4
PacMan-Maze	Image	CNN	<code>actor(x, y)</code> <code>enemy(x, y)</code> <code>goal(x, y)</code>	Plans the optimal action by finding an enemy-free path from actor to goal.	✓	✓	✓	31
CLUTRR (-G)	NL	RoBERTa	<code>kinship(rela, sub, obj)</code>	Deduces queried relationship by recursively applying learnt composition rules.	✓	✓	✓	8
	Query*	–	<code>question(sub, obj)</code>					
	Rule	–	<code>composition(r₁, r₂, r₃)</code>					
Mugen	Video	S3D	<code>action(frame, action, mod)</code>	Checks if events specified in NL text match the actions recognized from the video.	✓	✓	✓	46
	NL	DistilBERT	<code>expr(expr_id, action)</code> <code>mod(expr_id, mod)</code>					
CLEVR	Image	FastRCNN	<code>obj_attr(obj_id, attr, val)</code> <code>obj_rela(rela, o₁, o₂)</code>	Interprets CLEVR-DSL program (extracted from question) on scene graph (extracted from image).	✓	✓	✓	51
	NL	BiLSTM	<code>filter_expr(e, ce, attr, val)</code> <code>count_expr(e, ce), ...</code>					
VQAR	Image	FastRCNN	<code>obj_name(obj_id, name)</code> <code>obj_attr(obj_id, val)</code> <code>obj_rela(rela, o₁, o₂)</code>	Evaluates query over scene graphs (extracted from image) with the aid of common-sense knowledge base (KB).	✓			42
	KB*	–	<code>is_a(name1, name2), ...</code>					

Table 2. Characteristics of Scallop solutions for each task. Structured input which is not learnt is denoted by *. Neural models used are RoBERTa [Liu et al. 2019], DistilBERT [Sanh et al. 2019], and BiLSTM [Graves et al. 2013] for natural language (NL), CNN and FastRCNN [Girshick 2015] for images, and S3D [Xie et al. 2018] for video. We show the three key features of Scallop used by each solution: (R)ecursion, (N)egation, and (A)ggregation. †: For MNIST-R, the LoC is 2 for every subtask.

answer the question based on the image. We pick as baselines NS-VQA [Yi et al. 2018] and NSCL [Mao et al. 2019], which are *neurosymbolic methods* designed specifically for this task.

VQAR: Visual-Question-Answering with Common-Sense Reasoning. This task, like CLEVR, also concerns VQA but with three salient differences: it contains real-life images from the GQA dataset [Hudson and Manning 2019]; the queries are in a programmatic form, asking to retrieve objects in the image; and there is an additional input in the form of a common-sense knowledge base (KB) [Gao et al. 2019] containing triplets such as (giraffe, is-a, animal) for common-sense reasoning. The baselines for this task are NMNs [Andreas et al. 2016] and LXMERT [Tan and Bansal 2019].

6.2 RQ1: Our Solutions and Expressivity

To answer **RQ1**, we demonstrate our Scallop solutions to the benchmark tasks (Table 2). For each task, we specify the interface relations which serve as the bridge between the neural and symbolic components. The neural modules process the perceptual input and their outputs are mapped to (probabilistic) facts in the interface relations. Our Scallop programs subsequently take these facts as input and perform the described reasoning to produce the final output. As shown by the *features* column, our solutions use all of the core features provided by Scallop.

The complete Scallop program for each task is provided in the Appendix C. These programs are succinct, as indicated by the LoCs in the last column of Table 2. We highlight three tasks, HWF, Mugen, and CLEVR, to demonstrate Scallop’s expressivity. For HWF, the Scallop program consists of a formula parser. It is capable of parsing probabilistic input symbols according to a context free grammar for simple arithmetic expressions. For Mugen, the Scallop program is a *temporal*

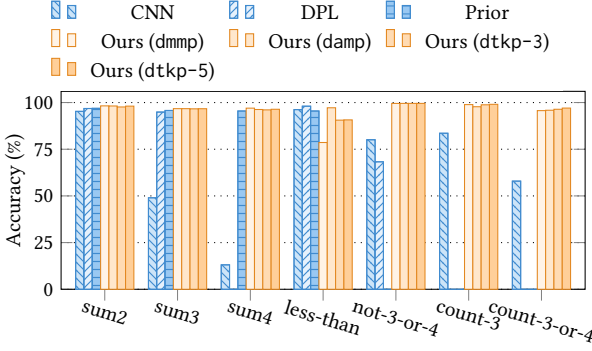


Fig. 15. MNIST-R suite accuracy comparison.

Method	Scallop			DQN
	dmmp	damp	dtkp-1	
Succ Rate	8.80%	7.84%	99.40%	84.90%
#Episodes	50	50	50	50K

Table 3. PacMan-Maze performance.

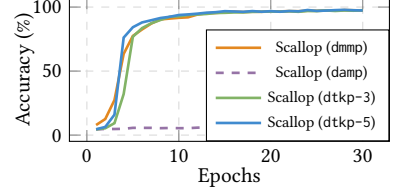


Fig. 16. HWF learning curve.

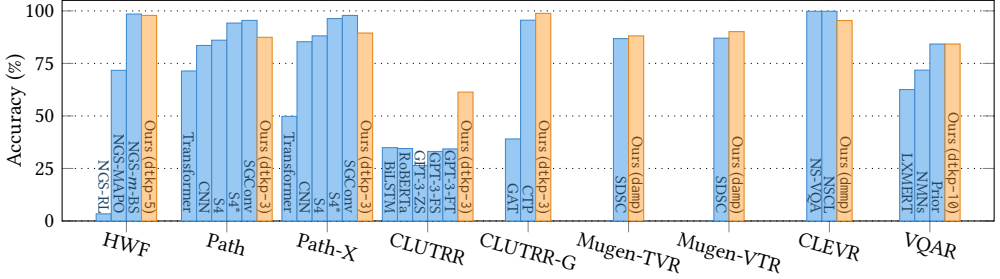


Fig. 17. Overall benchmark accuracy comparison. The best-performing provenance structure for our solution is indicated for each task. Among the shown tasks, dtkp performs the best on 6 tasks, damp on 2, and dmmp on 1.

specification checker, where the specification is extracted from NL text to match the sequential events excerpted from the video. For CLEVR, the Scallop program is an interpreter for CLEVR-DSL, a domain-specific functional language introduced in the CLEVR dataset [Johnson et al. 2017].

We note that our prior work [Huang et al. 2021], which only supports positive Datalog, cannot express 5 out of the 8 tasks since they need negation and aggregation, as indicated by columns ‘N’ and ‘A’. Moreover, HWF requires floating point support which is also lacking in our prior work.

Besides diverse kinds of perceptual data and reasoning patterns, the Scallop programs are applied in a variety of learning settings. As shown in Section 2, the program for PacMan-Maze is used in a *online representation learning* setting. For CLUTRR, we write integrity constraints (similar to the one shown in Section 3.2) to derive *semantic loss* used for constraining the language model outputs. For CLUTRR-G, learnable weights are attached to composition facts such as composition(FATHER, MOTHER, GRANDMOTHER), which enables to learn such facts from data akin to *rule learning* in ILP. For Mugen, our program is trained in a *contrastive learning* setup, since it requires to maximize similarity scores between aligned video-text pairs but minimize that for un-aligned ones.

6.3 RQ2: Performance and Accuracy

To answer **RQ2**, we evaluate the performance and accuracy of our methods in terms of two aspects: 1) the best performance of our solutions compared to existing baselines, and 2) the performance of our solutions with different provenance structures (dmmp, damp, dtkp with different k).

We start with comparing our solutions against selected baselines on all the benchmark tasks, as shown in Fig. 15, Table 3, and Fig. 17. First, we highlight two applications, PacMan-Maze and CLUTRR, which benefit the most from our solution. For PacMan-Maze, compared to DQN, we obtain a 1,000 $\times$ speed-up in terms of training episodes, and a near perfect success rate of 99.4%.